

# LIBRARY MANAGEMENT SYSTEM

## (REPORT)

### 1. PROJECT TITLE:

LIBRARY MANAGEMENT SYSTEM

---

### 2. PROJECT OUTCOME:

The Library Management System was successfully developed using **Java Spring Boot** and **MongoDB**. The project automates core library operations by managing students, staff, departments, books, library cards, and borrow records through a well-structured set of REST APIs.

#### **Achievements:**

- Implemented complete CRUD operations across all modules.
- Registered and managed student and staff records with department references using @DBRef relationships.
- Handled book inventory with real-time tracking of total and available copies.
- Issued library cards to students with unique card numbers and active/inactive status management.
- Automated book issuing with a default 14-day loan period and availability validation.
- Enabled book return with automatic increment of available copies.
- Supported book renewal with updated due dates and status tracking.
- Implemented search functionality to filter books by author.

- Handled exceptions globally using a centralized `GlobalExceptionHandler` returning proper HTTP error codes.

### 3. PROJECT SCREENSHOTS:

#### Screenshot 1 - Library Management System Initialization

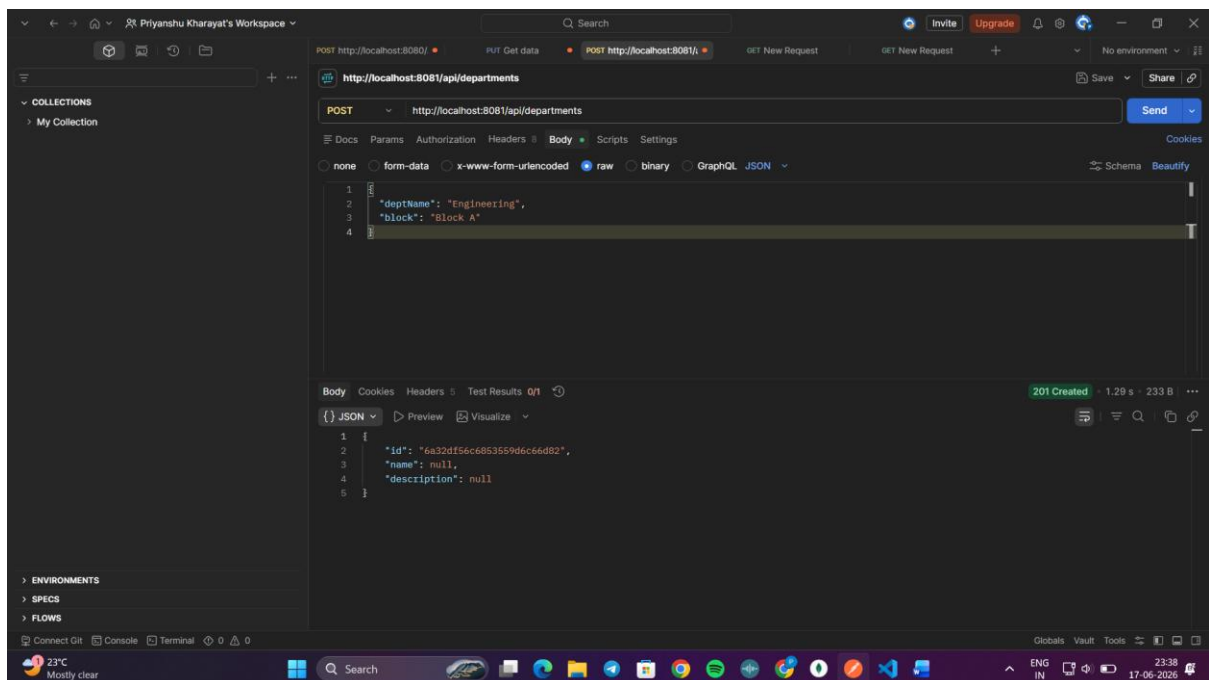
```

PS C:\Users\priya\Music\LIBRARY MANAGEMENT SYSTEM> .\mvnw.cmd spring-boot:run
2026-06-17T23:28:31.483+05:30 INFO 27088 --- [library-management] [main] c.l.m.LibraryManagementSystemApplication : Starting LibraryManagementSystemApplication using Java 25
2026-06-17T23:28:31.492+05:30 INFO 27088 --- [library-management] [main] c.l.m.LibraryManagementSystemApplication : No active profile set, falling back to 1 default profile:
"default"
2026-06-17T23:28:32.432+05:30 INFO 27088 --- [library-management] [main] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data MongoDB repositories in DEFAULT
mode.
2026-06-17T23:28:32.543+05:30 INFO 27088 --- [library-management] [main] s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 106 ms. Found
6 MongoDB repository interfaces.
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.apache.tomcat.jni.Library in an unnamed module (file:/C:/Users/priya/.m2/repository/org/apache/tomcat/embed/tomcat-embed-core/10.1
.19/tomcat-embed-core-10.1.19.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled
2026-06-17T23:28:33.065+05:30 INFO 27088 --- [library-management] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8081 (http)
2026-06-17T23:28:33.089+05:30 INFO 27088 --- [library-management] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.19]
2026-06-17T23:28:33.171+05:30 INFO 27088 --- [library-management] [main] o.a.c.c.c.[tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-06-17T23:28:33.172+05:30 INFO 27088 --- [library-management] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1
592 ms
2026-06-17T23:28:33.422+05:30 INFO 27088 --- [library-management] [main] org.mongodb.driver.client : MongoClient with metadata {"driver": {"name": "mongo-java
-driver|sync|spring-boot", "version": "4.11.1"}, "os": {"type": "windows", "name": "windows 11", "architecture": "amd64", "version": "10.0"}, "platform": "Java/Oracle Corporation/25.0.1
+8-115-27"}, created with settings: MongoClientSettings{readPreference=primary, writeConcern=WriteConcern{w=null, wtimeout=null ms, journal=null}, retryWrites=true, retryReads=true, readCo
ncern=ReadConcern{level=null}, credential=null, transportSettings=null, streamFactory=null, commandListeners=[], codecRegistry=ProvidersCodecRegistry{codecProviders=[ValueCodecPr
ovider(), BsonValueCodecProvider(), DBRefCodecProvider(), DBObjectCodecProvider(), DocumentCodecProvider(), CollectionCodecProvider(), IterableCodecProvider(), MapCodecProvider(), GeoLo
sonCodecProvider(), GridFSFileCodecProvider(), Jsr310CodecProvider(), JsonObjectCodecProvider(), BsonCodecProvider(), EnumCodecProvider(), com.mongodb.client.model.mql.ExpressionCodePr
ovider@63cd2cd2, com.mongodb.Jep395RecordCodecProvider@57a84fe, com.mongodb.KotlinCodecProvider@dee3370]}, loggerSettings=LoggerSettings{maxDocumentLength=1000}, clusterSettings={host
s=[localhost:27017], srvServiceName=mongodb, mode=SINGLE, requiredClusterType=UNKNOWN, requiredReplicaSetName='null', serverSelector='null', clusterListeners=[]}, serverSelectionTimeou
t='30000 ms', localThreshold='15 ms'}, socketSettings=SocketSettings{connectTimeoutMS=10000, readTimeoutMS=0, receiveBufferSize=0, proxySettings=ProxySettings{host=null, port=null, use
rname=null, password=null}}, heartbeatSocketSettings=SocketSettings{connectTimeoutMS=10000, readTimeoutMS=10000, receiveBufferSize=0, proxySettings=ProxySettings{host=null, port=null, us
ername=null, password=null}}, connectionPoolSettings=ConnectionPoolSettings{maxSize=100, minSize=0, maxWaitTimeMS=120000, maxConnectionLifeTimeMS=0, maxConnectionIdleTimeMS=0, maintainC
onInitialDelayMS=0, maintenanceFrequencyMS=60000, connectionPoolListeners=[]}, maxConnecting=2, serverSettings=ServerSettings{heartbeatFrequencyMS=10000, minHeartbeatFrequencyMS=500, se
rverListeners=[]}, serverMonitorListeners=[]}, sslSettings=SslSettings{enabled=false, invalidHostNameAllowed=false, context=null}, applicationName='null', compressorList=[], uuidRepr
esentation=JAVA_LEGACY, serverApi=null, autoEncryptionSettings=null, dnsClient=null, inetAddressResolver=null, contextProvider=null}
2026-06-17T23:28:33.467+05:30 INFO 27088 --- [library-management] [localhost:27017] org.mongodb.driver.cluster : Monitor thread successfully connected to server with desc
ription ServerDescription(address=localhost:27017, type=STANDALONE, state=CONNECTED, ok=true, minWireVersion=0, maxWireVersion=28, maxDocumentSize=16777216, logicalSessionTimeoutMinutes

```

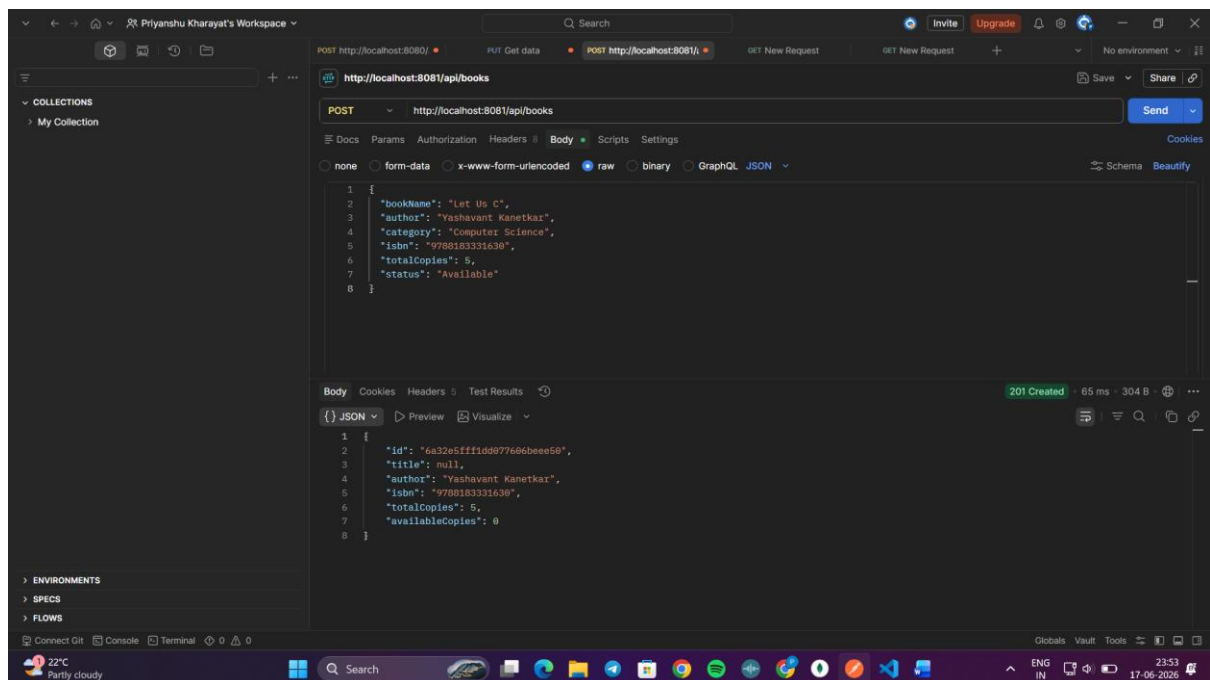
**Explanation:** The console output confirms the successful backend startup of the Library Management System. The Spring Boot application bootstrapped seamlessly, automatically configuring the necessary Spring Data MongoDB repositories. Furthermore, the logs indicate that the embedded Tomcat web server initialized on port 8081 and successfully established an active, stable connection to the local MongoDB cluster, preparing the system to handle data transactions.

## SCREENSHOT 2: API Endpoint Testing via Postman



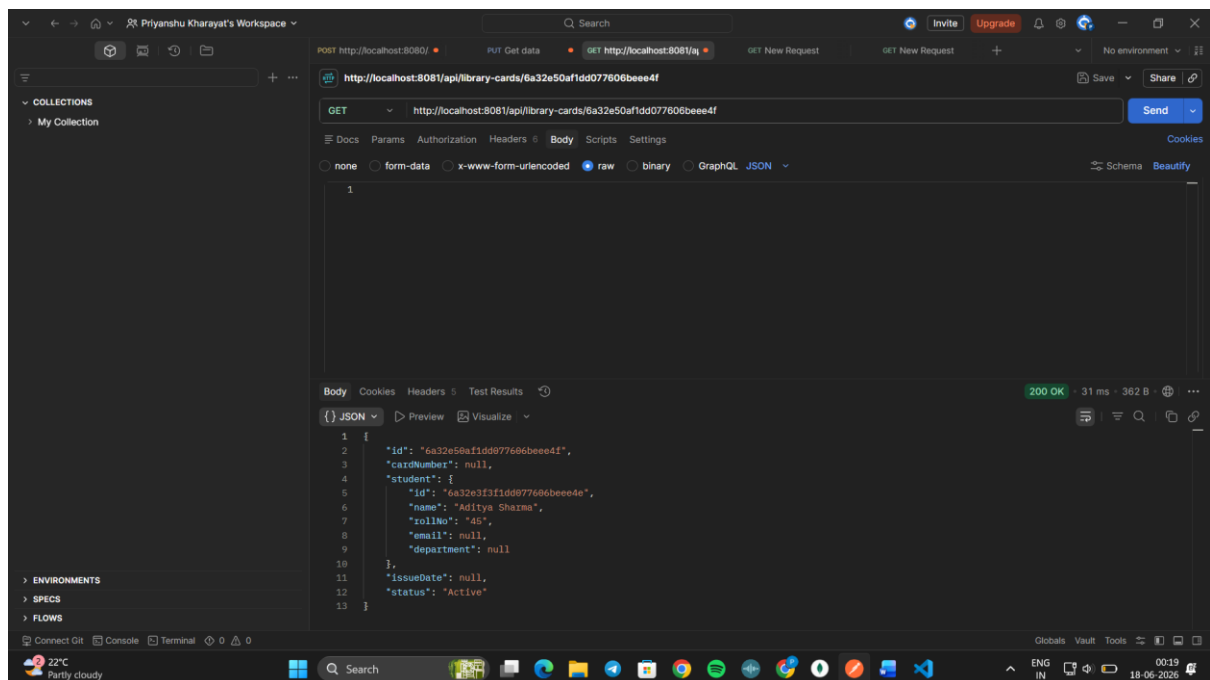
**Explanation:** This screenshot demonstrates the testing of the application's REST APIs using Postman. A POST request is being sent to the local server (<http://localhost:8081/api/departments>) to create a new department record with the details "Engineering" and "Block A" specified in the JSON payload. The server successfully processes the request, indicated by the 201 Created HTTP status code, and returns a JSON response containing the newly generated unique database ID for the entry. This confirms that the backend controller endpoints are active, correctly configured, and successfully communicating with the database to insert new data.

### Screenshot 3 - Book Registration API Testing via Postman



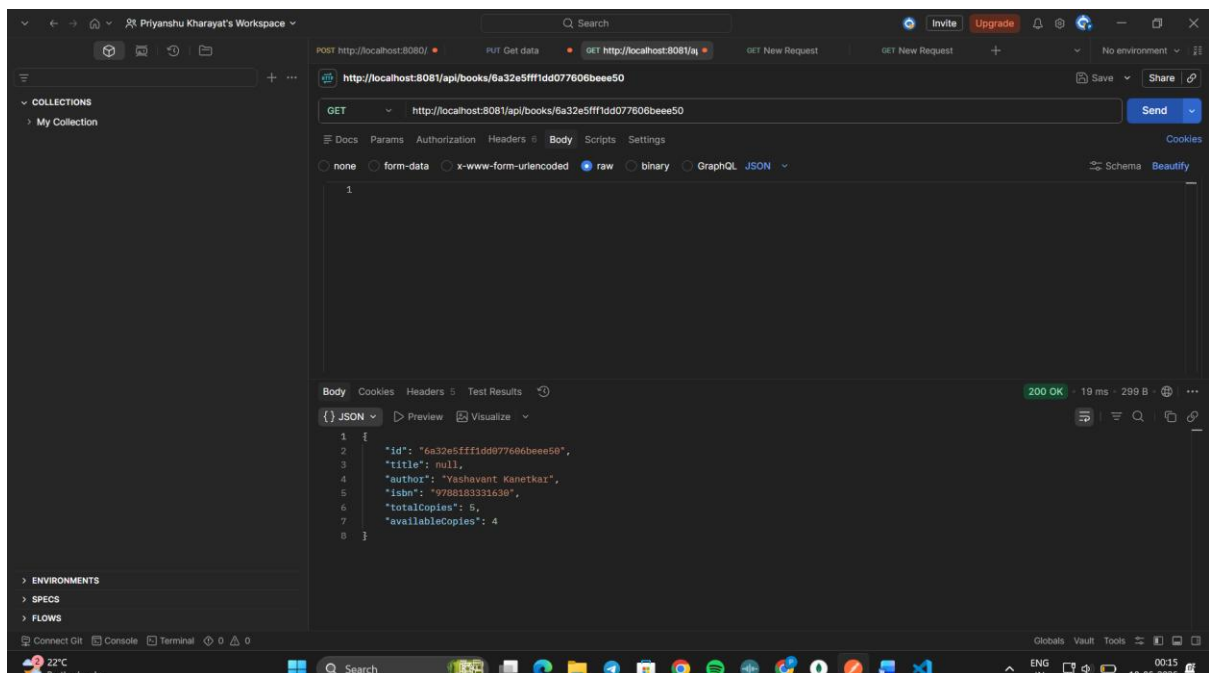
**Explanation:** This screenshot illustrates the testing of the library's inventory management functionality using Postman. A POST request is directed to the `http://localhost:8081/api/books` endpoint with a JSON payload containing the specific details for a new book entry ("Let Us C" by Yashavant Kanetkar). The system successfully handles the operation, which is verified by the 201 Created HTTP status code. Additionally, the response body reflects the newly persisted record along with an auto-generated unique database ID. This validates that the backend application is accurately receiving data, processing the request, and storing the library inventory records into the database.

## Screenshot 4- Retrieving Library Card Details via API



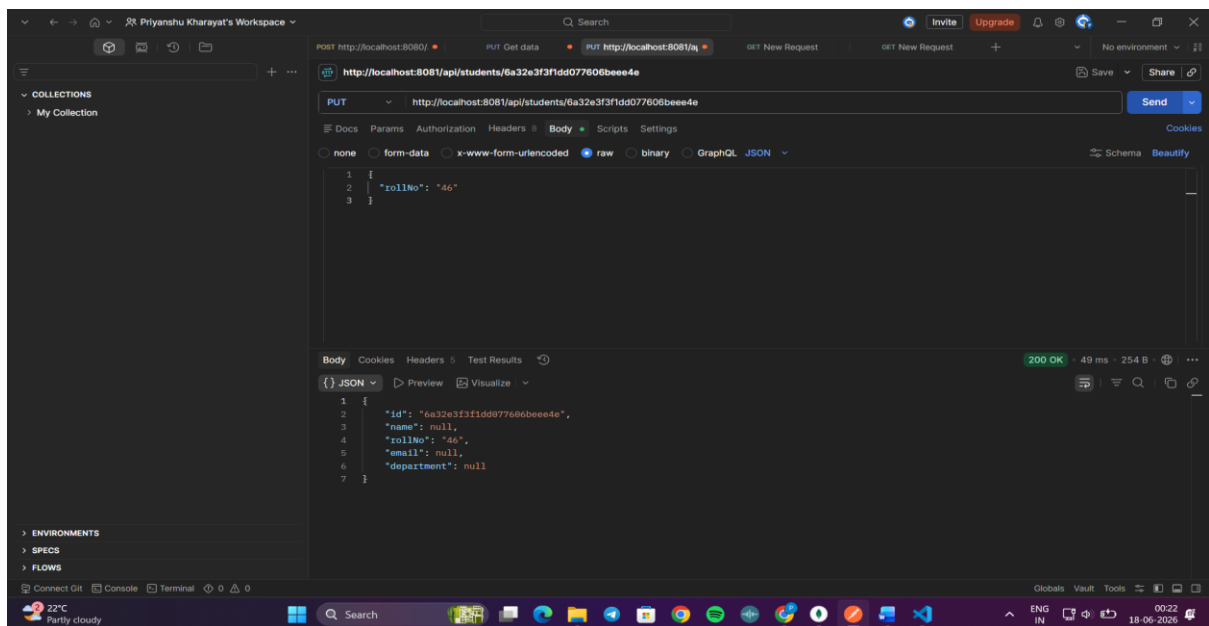
**Explanation:** This screenshot displays the successful retrieval of a specific record from the database using a GET request in Postman. The application is queried at the `http://localhost:8081/api/library-cards/{id}` endpoint to fetch the details associated with a particular library card. The 200 OK HTTP status code confirms that the server successfully received and processed the request. The returned JSON response reveals the requested database entry, which includes the active status of the card alongside nested information about the associated student (showing the name "Aditya Sharma" and roll number "45"). This effectively demonstrates that the backend's read operations and the data mapping between different entities (library cards and students) are functioning perfectly.

## Screenshot 5 - Retrieving Specific Book Details via API



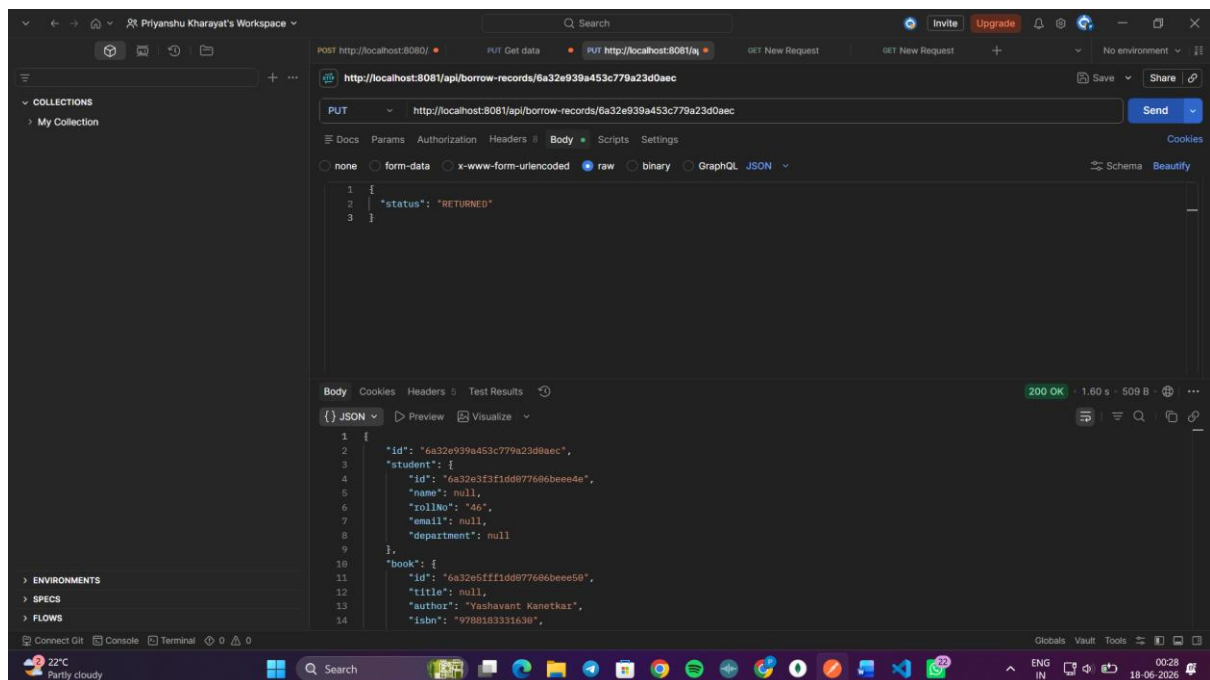
**Explanation:** This screenshot demonstrates a GET request executed in Postman to fetch the details of a specific book from the library database. The API endpoint (`http://localhost:8081/api/books/{id}`) is queried using the book's unique identifier. The server successfully processes the request, indicated by the 200 OK HTTP status code. The returned JSON payload displays the stored information for the requested book, including the author ("Yashavant Kanetkar"), the ISBN, and its current inventory status—showing 5 total copies with 4 currently available. This verifies that the backend application is accurately retrieving and serving specific inventory records from the database upon request.

## Screenshot 6 - Updating Student Records via API



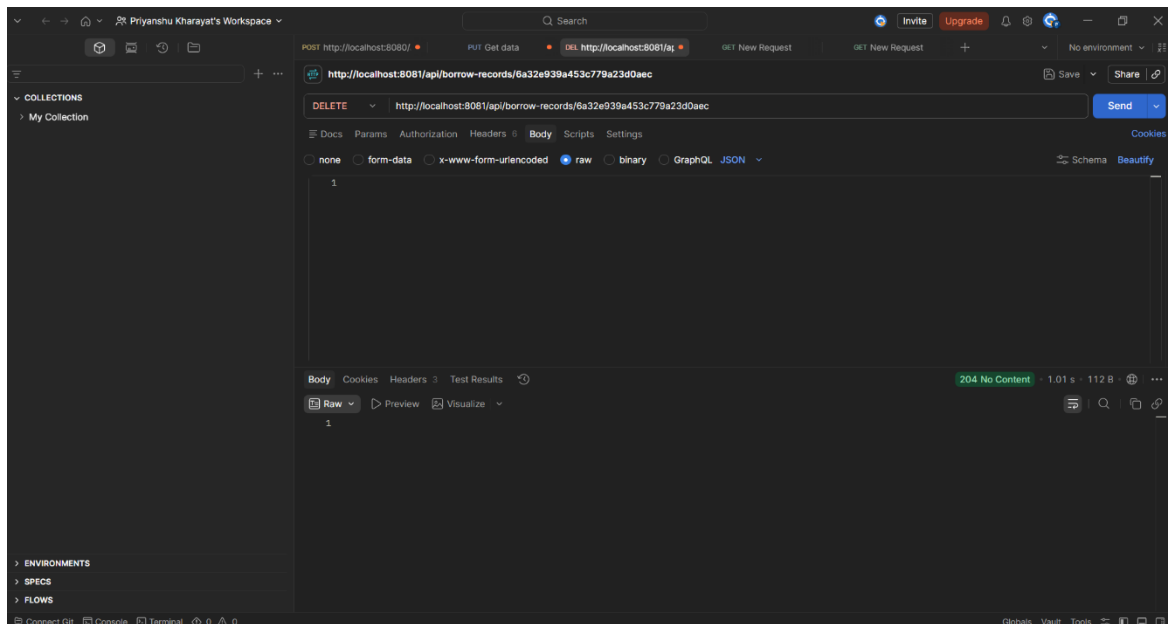
**Explanation:** This screenshot illustrates the functionality to update existing database records using Postman. A PUT request is directed to the `http://localhost:8081/api/students/{id}` endpoint to modify a specific student's profile. The JSON payload sent in the request body specifies an update to the student's roll number, changing it to "46". The system successfully processes this modification, as confirmed by the 200 OK HTTP status code. Furthermore, the returned JSON response body displays the updated student record reflecting the new roll number, proving that the backend controller properly handles data update commands and effectively persists those changes to the database.

## Screenshot 7 - Updating Borrow Record Status via API



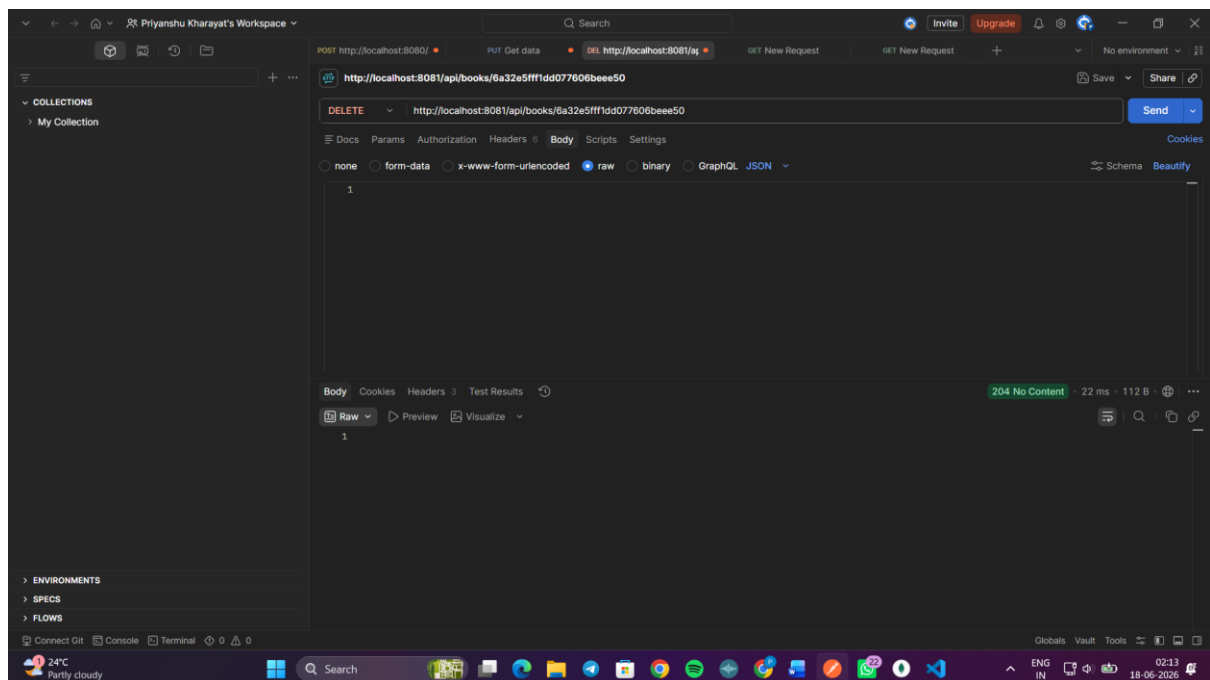
**Explanation:** This screenshot shows the process of updating a library transaction using a PUT request in Postman. The request is sent to the `http://localhost:8081/api/borrow-records/{id}` endpoint to modify an existing borrow record, specifically updating its status to "RETURNED" via the JSON payload. The 200 OK HTTP status code indicates that the server successfully processed the update. Additionally, the detailed JSON response confirms the transaction by displaying the linked data, including the student's roll number ("46") and the returned book's details (author "Yashavant Kanetkar"). This proves the backend effectively handles complex data relationships and successfully updates transaction states within the database.

## Screenshot 8 - Verifying Record Deletion via API



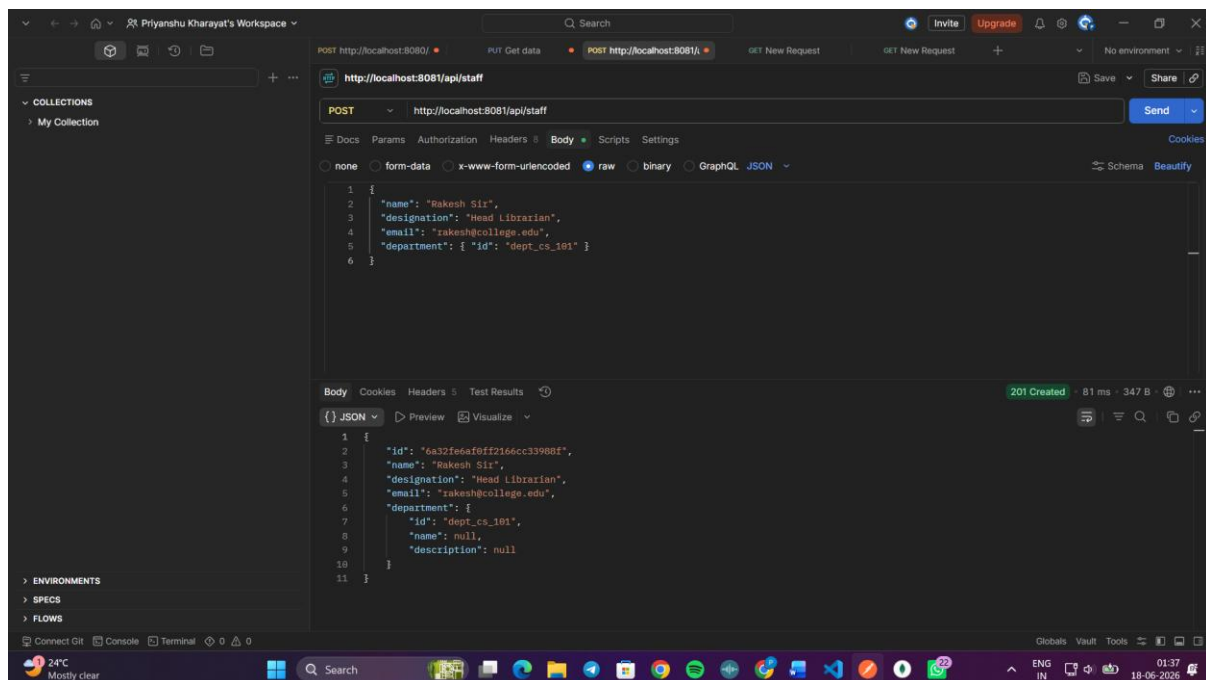
**Explanation:** This image captures another instance of testing the database removal operations using Postman. By sending a DELETE request to the `http://localhost:8081/api/borrow-records/{id}` endpoint, the system is instructed to erase a specific borrowing transaction. The successful execution of this command is validated by the 204 No Content HTTP response code shown on the lower right. This standard REST response confirms that the backend controller successfully processed the deletion and, as expected, returned an empty payload since the target record has been permanently removed from the database.

## Screenshot 9 - Deleting a Book Record via API



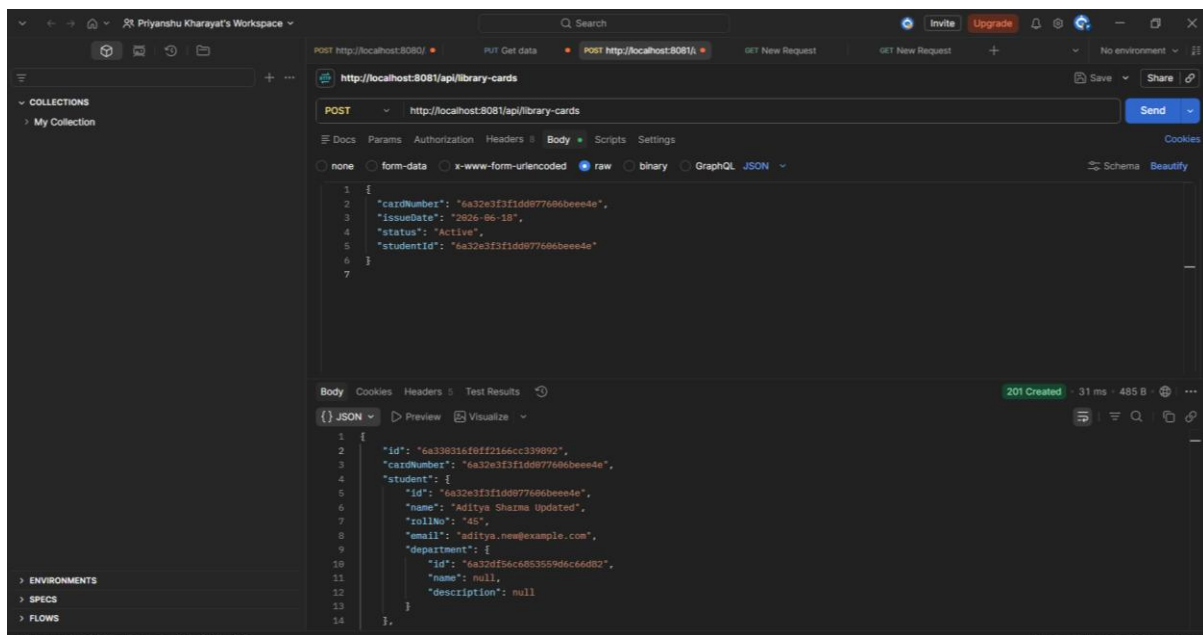
**Explanation:** This screenshot captures the process of removing a specific book from the library's inventory system using Postman. A DELETE request is sent to the `http://localhost:8081/api/books/{id}` endpoint, utilizing the book's unique database identifier to target the correct record. The system successfully executes the removal, which is verified by the 204 No Content HTTP status code. As expected for a standard REST API deletion, the server returns an empty response body, confirming that the backend controller successfully processed the command and permanently erased the targeted book record from the database.

## Screenshot 10 - Registering Staff Details and Many-to-One Mapping via API



**Explanation:** This screenshot illustrates the addition of a new staff member to the system using Postman, while specifically demonstrating a **Many-to-One** database relationship. A POST request is directed to the `http://localhost:8081/api/staff` endpoint with a JSON payload containing the staff details, including the name "Rakesh Sir" and the designation "Head Librarian." Crucially, the payload links this staff member to a specific department (`"id": "dept_cs_101"`), showcasing a Many-to-One association where multiple staff members can belong to a single department. The server successfully processes the request, confirmed by the 201 Created HTTP status code. The response body verifies that the backend controller effectively handles the data insertion and correctly persists this relational mapping in the database.

## Screenshot 11 - Creating Library Card and One-to-One Mapping via API



**Explanation:** This screenshot demonstrates the generation of a new library card using a POST request in Postman, specifically highlighting a **One-to-One** database relationship. The request is sent to the `http://localhost:8081/api/library-cards` endpoint, with the JSON payload defining the card's active status, issue date, and linking it to a specific `studentId`. This enforces a strict One-to-One association, ensuring that each library card is uniquely bound to one individual student. The server successfully processes the creation, indicated by the 201 Created HTTP status code. The returned JSON response validates this mapping by displaying the newly generated card details alongside the fully nested data of the associated student ("Aditya Sharma Updated"), confirming the backend properly handles and persists this One-to-One entity relationship.

---

#### 4. HTTP Methods Used in REST APIs:

##### **GET Method:**

The GET method is used to retrieve or fetch data from the server without modifying anything. For example, fetching all books, searching books by author, retrieving a specific student by ID, or getting all borrow records.

##### **POST Method:**

The POST method is used to create and store new data on the server. For example, registering a new student, adding a book, issuing a library card, or creating a borrow record when a book is issued.

##### **PUT Method:**

The PUT method is used to update existing records in the database. For example, updating student or staff details, changing the status of a library card, returning a borrowed book, or renewing a book with a new due date.

##### **DELETE Method:**

The DELETE method is used to permanently remove data from the server. For example, deleting a student, staff member, book, library card, or borrow record.

These HTTP methods form the foundation of CRUD operations:

- Create → POST
  - Read → GET
  - Update → PUT
  - Delete → DELETE
-

## 5. Tools Used

| Tool            | Purpose                 | Version |
|-----------------|-------------------------|---------|
| Java            | Programming Language    | JDK 17  |
| Spring Boot     | Backend Framework       | 3.2.4   |
| MongoDB Compass | Database GUI            | Latest  |
| Postman         | API Testing             | Latest  |
| VS Code         | Development Environment | Latest  |

## 6. Project Summary

The Library Management System is a backend application developed using Java Spring Boot and MongoDB. The system manages all core library operations through REST APIs and supports student registration, staff management, department organization, book inventory tracking, library card issuance, and borrow record management including book issue, return, and renewal. MongoDB is used as the NoSQL database running locally on port 27017, while Postman is used for API testing and collection management. The project follows a clean layered architecture — Controller, Service, Repository, and Model — and demonstrates database integration using Spring Data MongoDB, @DBRef for relational mapping between entities, and centralized exception handling for consistent error responses across all endpoints.

---

## 7. Conclusion

The Library Management System successfully digitizes and automates the typical manual processes involved in running a library. The project demonstrates practical implementation of Spring Boot REST APIs with MongoDB, covering all real-world requirements such as tracking book availability, managing member records, and handling the complete lifecycle of a borrowed book — from issuance to return or renewal. Features like automatic copy count adjustment on book issue and return, library card status management, and author-based book search add genuine usability to the

system. The project establishes a solid and scalable backend foundation that can be further enhanced in the future with features such as fine calculation for overdue books, email notifications on due dates, role-based access control, and a frontend interface for library staff and students.

